# Tic-Tac-Toe in C: Presentation Notes

Nov 9, 2026 · @Eligh

Notes for presenting `tictactoe.c`, a two-player terminal game whose per-player scoreboard survives closing the program: talk outline, live demo script, code details, and answers to likely questions.

## Talk outline

About 7 minutes plus questions; trim the code tour first if you run short.

| Step | Time | What to cover |
|---|---|---|
| 1. Intro | 0:30 | A two-player Tic-Tac-Toe in C for the terminal. Players enter names, and the scoreboard remembers each person after the program closes. |
| 2. Menu overview | 0:30 | `main` has three cases: 1) play game, 2) check scoreboard, 3) close game. |
| 3. Live demo | 2:00 | Follow the demo script below: two games, an invalid input, a duplicate name, the scoreboard, then restart to prove saving works. |
| 4. Code tour | 2:30 | Constants and structs, helper functions, game logic, then `main` at the very end. Use the function table below. |
| 5. Design details | 1:00 | Saved file format, name matching, safe input handling. |
| 6. Limits and next steps | 0:30 | Local two-player only, no computer opponent, scoreboard unsorted. |
| 7. Q&A | rest | Use the question list below. |

## Live demo script

Start from a fresh folder with no `scores.txt` so the first scoreboard check is empty. Every input below was run against the finished program.

1. Compile and run: `gcc -Wall -o tictactoe tictactoe.c` then `./tictactoe`.
2. Type `2`. The scoreboard says "No games played yet."
3. Type `1`, then `Alice` (X) and `Bob` (O). At Alice's first move type `abc` to show the input check ("That's not a number"), then play `1, 4, 2, 5, 3`. Alice wins as X with the top row.
4. Type `1`, then `Alice` for both names (or `Alice` and `alice`). The game refuses: the two players need different names.
5. Enter `Alice` and `Bob` again and play `1, 4, 2, 5, 7, 6`. Bob wins as O with the middle row. Point out that his win is counted in the O column, not the X column.
6. Optional draw: `Alice` and `Bob` again, then play `5, 1, 2, 8, 3, 7, 4, 6, 9`. The board fills with no winner and the draw total goes up.
7. Type `2`. Alice shows 1 win as X and 0 as O; Bob shows 0 as X and 1 as O; draws are 1 if you played the optional game.
8. Type `3` to close. Then run `cat scores.txt` to show the saved file.
9. Run `./tictactoe` again and type `2`. The same scores appear, which proves they persist.

## How the code works

The file reads top to bottom: constants, two structs, helper functions, game logic, the menu, and finally `main`. Every function has a comment block above it explaining its purpose, parameters, and return value.

- Constants: `BoardSize` (3), `ScoreFile` (scores.txt), `MaxUserIn` (64), `MaxName` (32), `MaxPlayers` (100).
- `Player` struct: a name plus `winsAsX` and `winsAsO`.
- `Scoreboard` struct: an array of `Player` records, how many there are, and the total draws.

| Function | What it does | Talking point |
|---|---|---|
| `readLine` | Reads one line with `fgets` and strips the newline; returns 0 at end of input. | Safer than `scanf`: bad input can't get stuck and loop forever. |
| `trimSpaces` | Removes spaces from both ends of a name. | Stops "Bob " and "Bob" becoming two players. |

| Function | What it does | Talking point |
|---|---|---|
| `namesMatch` | Compares two names ignoring upper/lower case. | "bob" and "Bob" are the same person. |
| `loadScores` | Reads `scores.txt` at startup into the scoreboard. | Missing or unreadable file means a fresh, empty scoreboard. |
| `saveScores` | Writes the scoreboard to `scores.txt`. | Called after every finished game and on exit. |
| `findPlayer` | Looks up a name and returns its index, or -1. | Linear search is fine for at most 100 players. |
| `findOrAddPlayer` | Returns a player's index, creating a new record if needed. | New players start at 0 wins as X and 0 as O. |
| `showScoreboard` | Prints one row per player: name, wins as X, wins as O, then total draws. | The format asked for: name, X wins, O wins. |
| `initBoard` | Fills the 3x3 board with empty spaces. | Board is a `char` 2D array. |
| `printBoard` | Draws the board; empty squares show their number 1-9. | Players just type the number they want. |
| `PlayerMove` | Places a mark if the square number is 1-9 and empty; returns 1 or 0. | The one place move rules are enforced. |
| `checkWinner` | Checks 3 rows, 3 columns, and 2 diagonals; returns 'X', 'O', or ' '. | 8 possible winning lines in total. |
| `isBoardFull` | Returns 1 if no empty squares remain. | Full board with no winner means a draw. |
| `getPlayerName` | Asks for a name until it is non-empty and under 32 characters. | Input validation in a loop. |
| `choosePlayers` | Asks for the X and O names, rejects identical names, finds or creates both records. | Players pick who is X each game. |
| `getPlayerMove` | Asks the current player for a square until the move is valid. | Rejects letters and taken squares. |

| Function | What it does | Talking point |
|---|---|---|
| `playGame` | Runs one full game, announces the result, updates scores, and saves. | X always goes first; turns alternate until a win or a draw. |
| `showMenu` | Prints the three menu options. | Keeps `main` short. |
| `main` | Loads scores, then loops on a `switch`: 1 play, 2 scoreboard, 3 save and quit. | Placed last in the file, as required. |

## Design details worth mentioning

- Scores are saved as plain text. The first line is the draw total and each later line is one player: wins as X, wins as O, then the name. The file is rewritten after every finished game, so closing the program, or even a crash, loses at most the game in progress.
- Scores belong to people, not to the marks. A player's X wins and O wins are counted separately, so the same person can appear in either role in any game.
- Names are matched ignoring case and surrounding spaces, and the first spelling typed is the one stored.
- Input is handled defensively: `fgets` for every line, `strtol` with a check for leftover characters on move entry, and name validation (not empty, under 32 characters).
- A win is checked before a full board, so a winning ninth move counts as a win rather than a draw.
- Closing input mid-game (Ctrl+D) abandons that game and records nothing.

Example `scores.txt` after the demo:

```
Draws: 1
1 0 Alice
0 1 Bob
```

# Q&A

| Question | Answer |
| --- | --- |
| How do scores survive closing the program? | `saveScores` writes them to `scores.txt` after every game and on exit; `loadScores` reads that file when the program starts. |
| What if `scores.txt` is missing or damaged? | Missing means a fresh empty scoreboard. If the first line is unreadable the scoreboard starts empty, and damaged player lines are skipped. The next saved game rewrites the file. |
| How do I reset the scoreboard? | Delete `scores.txt`. |
| How does it detect a winner? | `checkWinner` checks the 3 rows, 3 columns, and 2 diagonals for three matching non-empty marks. |
| How does it detect a draw? | After each move it checks for a winner first, then whether the board is full. Full with no winner is a draw, so a winning last move still counts as a win. |
| What happens if someone types a letter or picks a taken square? | `getPlayerMove` uses `strtol` and rejects anything that isn't purely a number, and `PlayerMove` rejects numbers outside 1-9 or squares already used. The player is asked again. |
| Why `fgets` instead of `scanf`? | `scanf` leaves bad input in the buffer, which can cause endless loops. `fgets` takes the whole line every time and the code parses it safely. |
| Can both players use the same name? | Not in one game; the program asks again. Across games, the same name (any capitalization) is the same person. |
| Why track wins as X and wins as O separately? | X always moves first, so the roles are not equal. Separate columns show how each person does in each role. |

| Question | Answer |
| --- | --- |
| How many players can it store? | Up to 100 players with names up to 31 characters. If the scoreboard is full, a new name is refused and the game is cancelled. |
| Why is `main` at the bottom? | In C a function must be defined or declared before it is used. Putting `main` last means no forward declarations are needed. |
| What happens if you type `1abc` at the menu? | The menu uses `atoi`, which reads the leading digits, so it behaves like `1`. This is a known limitation; switching to `strtol` as in the move entry would fix it. |
| Can two computers share one scoreboard? | No. It is one local terminal game, and two copies running at once could overwrite each other's saves because there is no file locking. |
| How did you test it? | Compiled with `-Wall -Wextra` with no warnings, then ran a win as X, a win as O, a draw, invalid input, duplicate names, and a restart to confirm scores persisted. |
| What would you add next? | A computer opponent, a sorted scoreboard, a rematch option, and safer menu parsing. |